

Projeto Final COBOL - Documento de Arquitetura

Cooperativa Financeira Alfa

1. Objetivo

Este documento descreve a arquitetura escolhida para o projeto de modernização da Cooperativa Financeira Alfa. A proposta é permitir que uma aplicação moderna com interface web e API consiga utilizar um fluxo legado em COBOL, mantendo a integração com banco DB2.

O projeto demonstra uma modernização gradual: o COBOL continua participando do fluxo principal, enquanto a interface, a API, os testes e a integração com banco são organizados em tecnologias atuais.

2. Arquitetura escolhida

A arquitetura escolhida foi uma integração em camadas:

Razor Pages -> API .NET -> COBOL -> Helper C# -> ODBC -> DB2

Essa escolha permite separar responsabilidades e manter o COBOL como componente central do processamento legado. A API funciona como ponte entre a tela moderna e o programa COBOL. O Helper C# realiza a comunicação com o DB2.

Camada	Responsabilidade principal
Razor Pages	Interface visual para consultar, cadastrar e atualizar clientes.
API .NET	Recebe requisições da tela, gera entrada para o COBOL e interpreta a saída final.
COBOL	Valida a operação e centraliza o fluxo legado da solução.
Helper C#	Faz a ponte entre COBOL e DB2 por meio de arquivo e ODBC.
DB2	Persiste os dados dos clientes.
Monitor COBOL	Exibe a execução do fluxo em uma janela de

Camada	Responsabilidade principal
	Prompt de Comando para apresentação.

3. Componentes utilizados

3.1 Razor Pages

- Responsável pela interface visual do usuário.
- Permite consultar cliente, cadastrar cliente, confirmar dados antes do cadastro e atualizar contato.
- Facilita a apresentação do projeto porque o fluxo pode ser demonstrado por uma tela web.

3.2 API .NET

- Recebe as requisições HTTP enviadas pela tela.
- Monta o arquivo runtime/entrada.txt em formato fixo para o COBOL.
- Executa o programa COBOL e lê runtime/saida.txt.
- Retorna o resultado em JSON para a interface.

3.3 COBOL

- Lê o arquivo runtime/entrada.txt.
- Valida a operação e os dados principais.
- Gera runtime/helper-entrada.txt para o Helper.
- Chama o Helper C# e lê runtime/helper-saida.txt.
- Gera runtime/saida.txt com o retorno final para a API.

3.4 Helper C#

- Recebe a operação enviada pelo COBOL.
- Consulta, cadastra ou atualiza clientes.
- Acessa o DB2 por ODBC.
- Retorna ao COBOL uma linha fixa compatível com o layout esperado.

3.5 DB2

- Banco relacional usado para persistência dos clientes.
- Banco utilizado no projeto: COOPDB.
- Executado em container Docker, usando o usuário db2inst1.

3.6 Monitor COBOL

- Janela de Prompt de Comando aberta junto com a API.
- Mostra cada consulta, cadastro ou atualização processada pelo COBOL.
- Exibe os arquivos de entrada, retorno do Helper/DB2 e saída final.

4. Justificativas técnicas

Decisão	Justificativa
Manter o COBOL no fluxo	O COBOL foi mantido para representar um cenário real de modernização, onde sistemas legados ainda concentram regras importantes de negócio.
Usar API .NET	A API cria uma camada moderna de comunicação, facilitando retorno em JSON e futuras integrações.
Usar Razor Pages	Razor Pages permite criar uma tela simples, funcional e adequada para demonstrar o projeto.
Usar Helper C#	O Helper simplifica a comunicação com o DB2 no ambiente local, mantendo o COBOL no controle do fluxo.
Usar arquivos fixos	Arquivos fixos são simples, compatíveis com COBOL e fáceis de verificar durante a apresentação.
Usar DB2	O DB2 mantém relação com ambientes corporativos e com o contexto de sistemas legados/mainframe.

5. Fluxo de execução da solução

5.1 Consulta de cliente

1. Usuário informa o código do cliente na tela.
2. Razor Pages envia a requisição para a API.
3. API monta o arquivo runtime/entrada.txt.
4. API executa o programa COBOL.
5. COBOL lê a entrada e valida a operação.
6. COBOL grava runtime/helper-entrada.txt.

7. COBOL chama o Helper C#.
8. Helper consulta o DB2.
9. Helper grava runtime/helper-saida.txt.
10. COBOL lê o retorno do Helper.
11. COBOL grava runtime/saida.txt.
12. API lê runtime/saida.txt e retorna JSON para a tela.

5.2 Cadastro de cliente

1. Usuário preenche nome, email e telefone.
2. Tela exibe popup de confirmação antes do cadastro.
3. Usuário confirma a operação.
4. API envia os dados para o COBOL por arquivo fixo.
5. COBOL valida os dados e chama o Helper.
6. Helper verifica duplicidade de email e cadastra no DB2.
7. COBOL gera a saída final.
8. API interpreta a resposta e a tela exibe o cliente cadastrado.

5.3 Atualização de contato

1. Usuário informa código, novo email e novo telefone.
2. Tela envia os dados para a API.
3. API monta o arquivo de entrada.
4. COBOL valida os dados.
5. Helper consulta e atualiza o cliente no DB2.
6. COBOL gera a saída final.
7. API interpreta o retorno e a tela mostra o resultado.

6. Arquivos principais do fluxo

Arquivo	Origem	Destino	Finalidade
runtime/entrada.txt	API	COBOL	Entrada principal da operação em formato fixo.
runtime/helper-entrada.txt	COBOL	Helper C#	Comando enviado pelo COBOL para consulta/cadastro/atualização.
runtime/helper-saida.txt	Helper C#	COBOL	Retorno do Helper após acessar o DB2.

Arquivo	Origem	Destino	Finalidade
runtime/saida.txt	COBOL	API	Saída final interpretada pela API e enviada para a tela.

7. Considerações finais

A solução final demonstra uma estratégia de modernização incremental. O sistema não substitui totalmente o legado; em vez disso, cria uma camada moderna ao redor dele, permitindo que o COBOL continue participando do processamento. A arquitetura também cria espaço para evoluções futuras, como autenticação, logs estruturados, APIs adicionais e integração direta com outros sistemas.